# Group Relative Policy Optimization (GRPO) for Video Scene Graphs

Complete Specifications, Results, and Code Directory Mapping for Plain G3 and G3 + PG

**svg2-killtest Project**
Karthikeya Anna

October 17, 2026

## 1 Overview & Motivation

Video Scene Graph Generation (VidSGG) requires detecting objects, tracking them, and predicting structured triplets $\langle \text{subject}, \text{predicate}, \text{object} \rangle$ grounded with temporal timestamps $[t_{\text{start}}, t_{\text{end}}]$.

The released baseline model, **TraSeR** (based on Qwen2.5-VL-3B with object slot tokens), encounters two major failure modes when generating entire graphs in a single sequence:

1. **Whole-graph salience bottleneck:** Generating all relations autoregressively causes the model to miss over 60% of interacting pairs due to token caps, loop-prevention truncation, and attention dispersion.
2. **Precision and hallucination trade-off:** Naive pair querying (adding candidate object pairs) introduces massive false-positive predicates (added-pair precision is only 11% on SVG2 and 4% on PVSG). Standard SFT suffers from format/convention bleed and looping.

To solve this, we developed a two-tier reinforcement learning strategy:

- **Plain G3:** Whole-graph GRPO that aligns the base model autoregressively on training videos, lifting precision, eliminating repetition loops, and increasing recall without formatting bleed.
- **G3 + PG:** A pair-factored architecture that combines G3's primary graph with a specialized pair-level GRPO adapter (**PG**) trained with explicit abstention rewards on negative candidate pairs.

## 2 Method 1: Plain G3 (Whole-Graph GRPO)

### 2.1 Formulation & Objective

Plain G3 optimizes the LoRA weights $\theta$ of the language model backbone via Group Relative Policy Optimization (GRPO). For each video $v$, a group of $G = 8$ rollouts $\{o_1, \ldots, o_G\}$ is sampled under temperature $T = 1.0$ and top-$p = 0.95$.

The surrogate loss per group is:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G} \min\left(\frac{\pi_\theta(o_i|v)}{\pi_{\text{old}}(o_i|v)} A_i,\ \text{clip}\left(\frac{\pi_\theta(o_i|v)}{\pi_{\text{old}}(o_i|v)}, 1-\epsilon, 1+\epsilon\right) A_i\right) + \beta D_{\text{KL}}(\pi_\theta \,||\, \pi_{\text{ref}}) \quad (1)$$

where advantage $A_i = \frac{r_i - \text{mean}(R)}{\text{std}(R)}$ is normalized strictly within the group of 8 candidate graphs of that video.

### 2.2 Reward Function

The reward $r(o_i, \mathrm{GT}_v)$ is evaluated via `engine/reward.py`:

$$r(o_i) = \frac{1}{|\mathrm{GT}_v|} \sum_{(s,p,o,\mathrm{span}) \in \mathrm{GT}_v} \max_{(s',p',o',\mathrm{span}') \in o_i} \mathbb{I}(s = s' \wedge o = o' \wedge \mathrm{norm}(p) = \mathrm{norm}(p')) \cdot \mathrm{tIoU}(\mathrm{span}, \mathrm{span}') \tag{2}$$

where temporal overlap tIoU $\geq 0.5$ provides soft fractional credit. Syntactic looping or malformed outputs receive zero reward.

### 2.3 Training Hyperparameters

- **Data:** TraSeR's own SVG2 training splits only (50% PVD / 50% VidOR mix; strictly no PVSG-train).
- **Optimization:** AdamW, learning rate $\eta = 3 \times 10^{-5}$, linear warmup for 20 steps, total 300 steps ($\sim 5.2$ hours).
- **LoRA Adapter:** Rank $r = 16$, $\alpha = 32$, applied to attention query/key/value projections in language layers.
- **Batch Size:** 1 video per gradient step (8 sampled graphs per step, $G = 8$).
- **Training Run:** Single deterministic training run (`traser-e1-g3`).

## 3 Method 2: G3 + PG (Pair-Factored Policy Optimization)

### 3.1 Two-Tier Architecture

1. **Base Stage (G3):** Generates the base scene graph $G_{\text{base}}$ for the full video.
2. **Candidate Proposer:** A 11-feature HistGradientBoosting classifier (`HistGBM`) scores all unmentioned object pairs based on bounding-box proximity, IoU, relative scale, and visual trajectory overlap. Top candidate pairs with predicted probability $\tau \geq 0.5$ are kept.
3. **Pair-Level Decoder (PG):** For each candidate pair $(a, b)$, the video features are sub-selected to only objects $a$ and $b$, and prompted with `"scene\_graph"`. The pair-level adapter decodes the predicate and temporal extent.
4. **Graph Merging:** Valid decoded pair relationships are deduplicated and merged into $G_{\text{base}}$ via `scripts/merge_pairs.py`.

### 3.2 PG Training & The Abstention Reward

The pair-level policy is trained via GRPO on pairs sampled from training videos. To combat hallucination on unlabelled pairs, the reward includes an explicit **abstention bonus**:

$$r_{\text{pair}}(y) = \begin{cases} \mathrm{RelationCredit}(y, \mathrm{GT}) & \text{if candidate pair is labelled in GT} \\ 0.5 & \text{if pair is an unlabelled hard-negative and model predicts “none”} \\ 0.0 & \text{if pair is an unlabelled hard-negative and model invents a relation} \end{cases} \tag{3}$$

This teaches the model to remain silent on unlabelled pairs while maintaining high recall on true interactions.

# 4 Benchmark Results

All metrics are computed using the strict per-span tIoU $\geq 0.5$ benchmark protocol with the Qwen3-8B lenient judge (SVG2 categories). Statistical significance ($^\dagger$) is determined via 2,000-sample paired bootstrap ($p < 0.05$).

**Table 1:** Benchmark evaluation across SVG2test (67 videos), VidOR-val (835 videos), and PVSG-val (62 videos). $^\dagger$ denotes statistically significant improvement over baseline TraSeR.

| Benchmark | Model Configuration | Precision (%) | Rel Recall (%) | Trip Recall (%) | Obj Acc (%) | F1 |
|---|---|---|---|---|---|---|
| **VidOR-val** | TraSeR Baseline (A0) | 26.8 | 9.2 | 6.4 | 78.8 | |
| | Plain G3 | **29.8**$^\dagger$ | **9.9**$^\dagger$ | 7.1$^\dagger$ | **79.1**$^\dagger$ | **1** |
| | **G3 + PG (Headline)** | **32.4**$^\dagger$ | **12.1**$^\dagger$ | **8.6**$^\dagger$ | **79.1**$^\dagger$ | **1** |
| **SVG2test** | TraSeR Baseline (A0) | 18.9 | 24.8 | 12.8 | 50.9 | |
| | Plain G3 | 18.8 | **30.5**$^\dagger$ | 14.1 | 49.9 | |
| | **G3 + PG (Headline)** | 16.8 | **36.9**$^\dagger$ | **16.4**$^\dagger$ | 49.9 | **2** |
| **PVSG-val** | TraSeR Baseline (A0) | 5.8 | 9.7 | 4.7 | 57.8 | |
| | Plain G3 | 6.2 | 10.1 | 5.1 | 57.3 | |
| | **G3 + PG (Headline)** | 5.9 | **14.1**$^\dagger$ | **6.4**$^\dagger$ | 57.3 | |
| *Multi-Sample Consensus Reference (Compute-matched ablation, 9 rollouts, vote3, §67):* | | | | | | |
| SVG2test | G3 + Vote3 + PG | **25.4**$^\dagger$ | **30.9**$^\dagger$ | 15.2$^\dagger$ | 50.9 | **2** |
| VidOR-val | G3 + Vote3 + PG | **40.0**$^\dagger$ | **11.6**$^\dagger$ | **8.5**$^\dagger$ | **79.1**$^\dagger$ | **1** |
| PVSG-val | G3 + Vote3 + PG | **8.4**$^\dagger$ | **13.1**$^\dagger$ | **6.5**$^\dagger$ | 57.8 | **1** |

**Key Takeaways:**

- **Plain G3 alone** significantly beats TraSeR on VidOR Precision, Recall, and F1 simultaneously (+1.1 F1$^\dagger$) and achieves a +5.7% recall gain on SVG2 without dropping precision.
- **G3 + PG** beats TraSeR on **Relation Recall and Triplet Recall across all three benchmarks with statistical significance**, and delivers a massive +3.9 F1 improvement on VidOR.

# 5 Code Directory Mapping

The implementation is self-contained within the repository. Table 2 details the exact file locations for each component.

**Table 2:** Codebase directory mapping for Plain G3 and G3 + PG.

| Component | Repository Path | Description |
|---|---|---|
| **1. Core Engines & Reward Functions** | | |
| GRPO Trainer | `engine/grpo.py` | Group relative policy update, advantage norm, KL penalty |
| Reward Metric | `engine/reward.py` | Graph-level soft tIoU reward & relation matching |
| Fast Slot Decoder | `engine/traser_fast.py` | Left-padded batching, GQA attention fix, LoopStop4 |
| Smoke Tests | `engine/smoke_grpo.py` | CPU/GPU smoke verification for GRPO forward/backward |
| **2. Training Configurations & Kernels** | | |
| G3 Kernel Config | `kaggle/e1/cfg_G3.py` | Hyperparameters, lr $3 \times 10^{-5}$, 300 steps, $G = 8$ |
| G3 Kernel Script | `kaggle/e1/G3/e1_g3.py` | Assembled Kaggle script for Plain G3 |
| GRPO Kernel Body | `kaggle/e1/grpo_part.py` | Per-step rollout sampling, advantage scoring, optimizer |
| PG Kernel Config | `kaggle/e1/cfg_PG.py` | Pair-level GRPO configuration (120 steps) |
| PG Kernel Script | `kaggle/e1/PG/e1_pg.py` | Assembled Kaggle script for Pair GRPO |
| PG Kernel Body | `kaggle/e1/pg_part.py` | Abstention reward loop and candidate pair decodes |
| **3. Pair Proposer, Merging & Evaluation Scripts** | | |
| Pair Candidate Ranker | `scripts/pair_feats.py` | Extracts 11 geometric spatio-temporal features |
| Pair Generation | `scripts/make_pairs.py` | HistGBM model inference to produce candidate pair lists |
| Pair Merging | `scripts/merge_pairs.py` | Integrates decoded candidate pairs into base G3 graphs |
| Deduplication | `scripts/dedup.py` | Cleans repeated text patterns and duplicate spans |
| Scoring Harness | `scripts/score_runs.py` | Computes P, R, and F1 across benchmark predictions |
| Bootstrap Significance | `scripts/bootstrap.py` | 2,000-fold paired bootstrap confidence intervals |
| Triplet Recall Tool | `scripts/boot_triplet.py` | Measures joint relation + object label correctness |
| **4. Outputs and Checkpoints** | | |
| G3 Predictions | `out/e1_g3/` | Full graph outputs on SVG2, VidOR, PVSG |
| PG Decodes | `out/e1_pg/` | Top-40 candidate pair decodes with PG adapter |
| G3 + PG Merged Graphs | `out/e1_g3_pg/` | Merged headline evaluation files |